

开题答辩·逐页口播提示卡

开题答辩·逐页口播提示卡

《基于大模型编码智能体的自进化 eBPF 网络入侵防御系统》·李兆曦·指导教师徐恪教授总时长约 13-15 分钟。下表按幻灯片顺序, 每页给关键词式提示 (瞄一眼即讲), 非逐字稿。

封面 0.5 - 各位老师好, 我是李兆曦, 指导教师徐恪教授 - 题目报一遍 → “下面从背景、问题、相关工作、内容方法、技术路线、创新点、计划成果汇报”

目录 5 - 一句带过:“汇报分八个部分”

背景：跨层歧义 → **off-path 攻击** 0.7 - TCP/IP 跨层交互有语义歧义 → 催生 **off-path(链外) 攻击** - 攻击者: 不在路径上、不窃听, 仅凭侧信道推 **序列号** → 劫持/重置/污染 - 例:challenge-ACK(CVE-2016-5696)、IPID、PMTUD、NAT/Wi-Fi - **CCS'25:PMTUD** 打破连接隔离 → 220 秒 / 七成成功率 / 50 网络中 38 脆弱 - 落点: 攻击面植根于“设计层面歧义”, 不断再生

背景：防御反而制造攻击面 0.5 - 关键规律: 很多新攻击面由防御自己引入 - challenge-ACK 限速 → 侧信道; 端口随机化 → 被反推 - 落点: 攻防同源、动态博弈 → 防御必须持续演化

背景：现有防御三大缺陷 0.5 - 零散 (各打各的补丁)/ 滞后 (被动响应)/ 依赖人工写内核代码 - 人工: 慢、易错、知识难沉淀 - 落点: 防御的“生产方式”仍是人工、低速、难累积

背景：两项技术成熟 0.5 - **eBPF**: 热加载、贴近协议栈、内置 **verifier** 做形式化验证 - **编码智能体**:Claude Code / Cursor / Codex / Devin / SWE-agent 已能自主编写、修改与调试代码 - 落点: 二者结合 → “让智能体持续维护内核防御”可行

核心研究问题 0.5 - 能否让编码智能体自主维护内核 eBPF 防御库, 使其持续进化? - 强调: 不是一次性合成, 而是长期自主运转 + 能力单调累积

关键挑战 (工程层面) 0.7 - **可靠性**: 内核代码若有缺陷 → 网络中断/误伤正常流量, 先验证安全、再上线, 异常可回滚 - **生成质量**: 智能体生成的 eBPF 须可编译、通过 verifier、有效拦截且不误伤正常流量 - **评测**: 见过/没见过的攻击都能防, 开销可接受

相关工作 (一): 协议攻防 + eBPF 防御 0.7 - 攻防: off-path 系列 + 检测 (SCENT/SCAD); 防御零散滞后 - eBPF 防御: Katran/Cilium/Falco/Tetragon、verifier(PREVAIL/K2)——**策略均人工写**

相关工作 (二): LLM 安全 + 方法论 0.5 - 安全: PentestGPT/Big Sleep; Kgent/KEN(最直接基线, 但一次性合成) - 方法论: 非梯度演化、FunSearch/AlphaEvolve、Darwin-Gödel Machine、探索(ICM/RND)、QD(Novelty/MAP-Elites)

定位对比表 0.5 - 对比 Kgent(一次合成)/ Tetragon · Falco(人工)/ 小模型 (权重) - 本文: 长期自主维护、自进化、知识沉淀为**可验证代码**

核心思路: 防御演化 0.7 - 一句话: 让 agent **持续维护与演化**一套内核防御代码; 防御沉淀为**可验证代码**, 不是模型权重 - 优势: 容量无界 / 抗遗忘 / 可解释 / 线速 / 可形式化验证

主要研究内容 (三项) 0.7 - **系统构建**: PacketScope/Guarder 上, 构建“生成 eBPF → 编译 → verifier → 沙箱 → 加载 → 读取遥测”的工具链闭环 - **写出可用且安全的 eBPF**: 生成质量 (可编译 / 通过 verifier / 不误伤) + 运行安全 (异常回滚) - **真实攻击防御 + 评测**: PMTUD/NAT/IPID 靶场, 度量检测率 / 误报 / 开销, 与人工规则、现有工具对比

系统分层架构 (看图讲) 0.7 - 三层: **控制面 · 智能体** (探索/合成/反思 + harness) → **可信验证面** (编译 → verifier → 沙箱 → 回滚) → **内核数据面** (XDP tail-call 防御链 + 遥测) - 左: 攻防靶场喂攻击样本; 右: **防御库/知识库**沉淀与采样 - 主流向下 (候选 eBPF → 验证通过 · 热加载); 运行时**反馈回灌**控制面再迭代

技术路线: 防御演化闭环 (五阶段)(看图讲) 1 - 探索 → 合成 → 验证 (失败 → 自修正) → 部署反馈 → 档案 → 回灌 - 一句话: 每轮把新攻击面 → 一段 verifier 背书的 eBPF 防御, 沉淀入库

可信保障: 四层 + 自动回滚 0.7 - 候选 → 静态沙箱 → 模板封装 → -Werror → **内核 verifier** (都是现成基础设施) - **创新在于接入智能体闭环**: 报错驱动**自修正**、沙箱重放把关、异常即**自动回滚**

工程基线: 已有开源系统 0.5 - PacketScope / Guarder: XDP 数据面、一键编译、热加载、Agent 可编程 eBPF 原型 + 312 测试 - 落点: 底座已具备, 闭环可快速搭

创新点 (三) 0.7 - **一套能跑的系统**:PacketScope/Guarder 上落地,agent 自动维护内核 eBPF 防御,可演示可复现 - **可靠流水线**: 生成 → 编译 → verifier → 沙箱 → 加载 → 回滚,智能体生成的 eBPF 能用、不致内核崩溃 - **真实攻击评测靶场**:PMTUD/NAT/IPID, 检出率 / 误报 / 开销,比人工规则与现有工具

工作计划 + 预期成果 0.7 - 计划: 下半年框架原型 → 年底闭环 + 实验 → 明年上半年评测 + 中期 → 撰写投稿 → 6 月答辩 - 成果: 学位论文; 开源系统 + **harness** + 评测基准; 必要时向 IETF/Linux 负责任披露

致谢 0.3 - 一句总结: 把攻击者视角反向用于防御,让智能体以“防御演化”自主维护内核 eBPF 防御库 - “恳请各位老师批评指正,谢谢!”

提示

- **加粗** = 必须说到的关键词; = 建议用时 (总约 13–15 分钟,时间紧时优先压缩“相关工作”与“工作计划”)。
- 看图的页 (架构、闭环、四层) 边指边讲,不要照读文字。
- 答辩提问预案见《演讲稿》文末 Q1–Q5。